

From next-token prediction to cheap critics and formal proof search

Project briefing · 28 September 2026. Results below are from completed experiment reports. The calibrated three-seed online comparison was still running at the latest reported checkpoint.

1. The question behind the project

Can an LLM’s existing internal representations support a useful critic cheaply enough to improve the cost of reinforcement learning?

An LLM already performs substantial computation to decide its next token. We investigate whether a small prediction head can reuse that computation to estimate whether an unfinished answer will eventually succeed. Our main application has been PPO, where this estimate helps decide which generated actions to reinforce.

The actor is **Qwen2.5-Math-1.5B base**. We began with GSM8K arithmetic word problems and moved to the **numeric-answer subset of MATH levels 4 and 5**. We have demonstrated useful value prediction and encouraging online learning. We have not established a universal advantage for LSTD, or superiority over a well-tuned PPO baseline across seeds.

2. One example connects language modelling and RL

Consider: **“Georgie has 5 avocados and receives 4 more. Each serving requires 3. How many servings can she make?”**

The answer is $(5 + 4) / 3 = 3$. The model writes its solution one token at a time. In RL terms:

RL concept	Meaning in this example
State	The question and everything written so far
Action	The next token
Policy / actor	Qwen’s probability distribution over next tokens
Transition	Append the chosen token to the answer
Terminal reward	1 for a correct final answer; otherwise 0 under the scoring policy
Value	Expected final reward if the current policy continues from this prefix

After “There are 9 avocados,” a critic might predict a high chance of eventual success. That is **not a certificate that this sentence is correct**. A correct prefix can still lead to a wrong answer; a confused prefix can be repaired later.

For binary outcomes, the ideal value is:

$$V^\pi(s) = \Pr(\text{eventual correct answer} \mid s, \pi)$$

We use the LLM’s **final normalized hidden vector at the last prefix token, before choosing the next token**. It summarizes the question and partial answer through the transformer’s nonlinear computation. It is not the vocabulary probability vector, and it contains no future answer tokens. The head predicts value from this vector.

A linear head therefore does not mean that the entire system reasons linearly: the expensive nonlinear feature computation has already happened inside Qwen.

3. How PPO, GRPO and our critics differ

PPO uses a critic to estimate values along an answer. GAE combines successive value predictions and observed rewards into advantages: was this continuation better or worse than expected? PPO then changes token probabilities with a clipped objective that limits individual update ratios. A conventional transformer critic learns from detached return targets; its optimizer persists across updates. Our implementation retains VERL’s PPO/GAE machinery. [VERL PPO documentation](#).

GRPO instead samples several answers to the same question and compares their rewards. If three avocado answers are wrong and one is correct, the successful answer receives a positive relative signal. It needs no learned critic. If every answer receives the same reward, that group supplies no outcome-based distinction between them.

Our PPO alternatives change the critic:

- **Ridge**: fit a regularized linear head directly to eventual outcomes. Every sampled prefix of a successful avocado answer receives target 1; every prefix of an unsuccessful answer receives target 0. Individual labels are noisy samples of the underlying success probability.
- **LSTD(λ)**: fit a linear value function using relationships between consecutive states, rewards and eligibility traces. The trace spreads information over preceding states. We accumulate a linear system and solve for its weights rather than iterating a neural-network optimizer.

With fixed features, suitable conditions and matched regularization, LSTD targets the corresponding empirical linear TD fixed-point equations. This is not a guarantee of the true value function. Our use is **state-value evaluation**, not the full action-value policy-improvement algorithm in the motivating [Least-Squares Policy Iteration paper](#).

In our complete episodic setup, with discount $\gamma=1$ and matched weighting, **LSTD(1) reduces to the same return-regression solution as ridge**. Smaller λ relies more on bootstrapping. LSTD’s λ is separate from GAE’s λ , which controls the actor’s advantage estimate.

In online linear-critic pilots, we capture detached features from the current actor, fit heads on fresh batches, and use question-level cross-fitting: an answer’s own outcome does not train the head predicting its values. PPO updates the actor; the next iteration captures its changed representations and fits new heads. The offline weights are not simply installed and left unchanged.

4. What the software does

Component	Responsibility
Slurm	Allocates cluster resources and runs jobs independently of the SSH session
vLLM	Generates answers efficiently
PyTorch and FSDP	Compute gradients and distribute model state across GPUs
VERL	Coordinates rollouts, rewards, log probabilities, advantages and RL updates
Math-Verify	Extracts and compares final mathematical answers
Our code	Builds datasets and prompts; integrates rewards; captures features; fits critics; runs comparisons and reports costs

Online comparisons generally use **two RTX A5000 GPUs**. Our contribution is the critic system and experimental controls, not a replacement implementation of every PPO component.

An early obstacle was reward extraction: correct prose answers received zero because they lacked the required formatting. Math-Verify recovered many of these false rejections. It still checks **final answers, not the reasoning**, and model-generated Python “output” is not evidence that code actually ran.

Offline critic studies exclude answers without usable verifier labels and report those exclusions. Online comparisons count ordinary parse failures and verifier timeouts as zero for every method, while recording their status. These policies define different evaluated populations. The independent reward audit remains incomplete.

5. What we learned from frozen-model experiments

First, short PPO and GRPO smoke runs established that generation, scoring, backpropagation and checkpoint saving worked. On 500 GSM8K questions, the base model solved **423**, and each trained model solved **424**. That one-answer gain was not convincing learning evidence; those early runs also had unequal sampling budgets.

We then froze the actor and generated **80,000 answers**: 4,000 training questions, 500 validation and 500 test questions, with 16 answers each. We retained **77,351** answers and cached causal prefix representations. Head fitting used random prefix lengths, including question-only states; splits kept all answers to one question together.

Two metrics must be distinguished:

- **Answer accuracy**: how often the LLM solves the math problem.
- **Critic accuracy**: how often a head predicts whether an answer will succeed. **Brier error** measures squared prediction error against binary outcomes; lower is better. Raw linear predictions need not lie in $[0,1]$.

Frozen GSM8K head	Test Brier ↓	Critic accuracy
Constant	0.23054	-
One-layer sigmoid head	0.19268	71.3%
Two-layer MLP	0.18888	71.8%
Ten-layer residual network	0.18977	71.6%
Direct ridge	0.19419	71.2%

Finding: a small head recovers useful value information, and additional depth offers only modest improvement here. The sigmoid head uses cross-entropy; ridge uses squared-error regression. Ridge fitting and tuning took about **32 seconds**, excluding generation and feature extraction. An initially undertrained linear head performed badly; increasing its fitting budget corrected that misleading result.

For matched LSTD comparisons, we used **20.9 million consecutive token transitions**. On GSM8K, LSTD(0) produced Brier **0.22889**, LSTD(0.99) **0.20222**, LSTD(0.999) **0.19751**, and ridge **0.19845**. A new generation seed on the same questions reproduced the small near-one- λ advantage.

Evaluation	LSTD Brier ↓	Ridge Brier ↓	Interpretation
New GSM8K answers, $\lambda=0.999$	0.19790	0.19887	Small repeatable difference on the same questions
Transfer to 148 hard MATH questions, $\lambda=0.999$	0.20232	0.20886	Larger transfer advantage
MATH-specific fitting, validation-selected $\lambda=1$	0.18566	0.18566	Identical fits; no LSTD advantage

The paired MATH transfer difference was **-0.00650**, with descriptive 95% interval **[-0.00887, -0.00411]**. Longer answers favored LSTD in that transfer experiment, but length is not a controlled measure of reasoning depth. MATH-specific fitting used **15,346 retained training answers and 11.7 million transitions**. Lower traces, including 0.85-0.95, performed substantially worse than ridge in follow-up transfer tests.

Lesson: LSTD is not inherently better than outcome regression. Long traces suit these sparse terminal rewards; reliable cheap value prediction is the stronger finding.

6. The important step: the critics helped an actor learn

We next compared four methods on MATH levels 4/5. Each received **30 updates × 64 answers = 1,920 training answers**, with the same base model, reward and decoding setup. All began at **52.3%** on the same 300-question greedy evaluation.

Method	Final answer accuracy	Gain	Training GPU-hours ↓
Conventional PPO	52.3%	0.0 pp	2.611
PPO-ridge	58.0%	+5.7 pp	1.809
PPO-LSTD(0.99)	60.0%	+7.7 pp	1.821
GRPO	60.3%	+8.0 pp	1.775

LSTD-PPO corrected **39** base-model mistakes and introduced **16**, gaining **23 correct answers overall**. Its fitting took only **81.5 seconds across 30 updates**. Measured training used roughly **30% fewer GPU-hours** than this conventional PPO configuration. Costs include feature transport, fitting, validation and checkpoints; they exclude setup and initial/final test evaluation.

This is genuine evidence that the small-critic system can support useful PPO learning. It is **one-seed pilot evidence**, not proof that LSTD beats ridge or GRPO. Nor does it establish savings against an optimally tuned conventional critic. Architecture, targets, persistence and cross-fitting differ between critic systems, so the result cannot be attributed solely to the solver.

Two extensions did not earn a place in the main pipeline:

- **Buffer:** with a frozen feature encoder, LSTD scored 57.7% without the buffer and 57.3% with it; ridge scored 56.7% and 56.3%. The buffer increased cost by roughly 10% and 5%, respectively. One-answer differences do not prove harm, but no benefit was demonstrated. Frozen features prevent representation drift; they do not make old-policy outcomes on-policy. The buffer is paused.
- **Explicit planning:** the final staged screen scored **19.25%** for the control versus **14.5%** for facts → plan → solution → final answer, using **582 versus 773 tokens** on average. At least one section hit its budget in **38.8% versus 98.8%** of attempts. Formatting failures, forced boundaries and instruction-following problems undermined the approach. This does not establish a 1.5B parameter ceiling or show that planning cannot help a different model.

7. Where the current comparison stands

We strengthened the conventional PPO baseline before drawing stronger conclusions. Three configurations passed numerical and critic-training health checks. On 200 validation questions, from a **51.5%** starting accuracy:

Critic configuration	Final validation accuracy
Learning rate 1e-5, two epochs	49.5%
Learning rate 5e-5, two epochs	52.5%
Learning rate 1e-5, four epochs	54.5%

The last configuration was selected using validation only. Calibration cost **8.86 training GPU-hours**. Reduced training-target error confirms fitting activity, not held-out value accuracy.

The ongoing comparison runs **PPO, PPO-ridge, PPO-LSTD(0.99) and GRPO for 60 updates each across three seeds**, with no buffer and matched actor settings. At the last update it had reached LSTD on the third seed. One completed GRPO run improved from **52.3% to 56.3%**; the full aggregate is not yet available.

The remaining limits are explicit: repeatedly inspected test questions, incomplete independent reward auditing, unknown pretraining overlap, numeric-only MATH coverage and short training budgets. A stronger internal comparison is underway; an untouched external benchmark remains necessary.

8. Why pivot next to AlphaProof-lite?

The case is to test a new use of a promising cheap critic - not to claim PPO failed. We have established useful prefix information and encouraging online learning, while the incremental advantage of LSTD itself remains mixed.

AlphaProof couples language-model proposals, formal proof search and reinforcement learning in Lean. Verified proofs provide reliable feedback. Our proposed “lite” version borrows that principle at a manageable scale; it is not a reproduction of the full system. [DeepMind’s AlphaProof description](#).

Return to the avocado example. Instead of accepting a plausible paragraph ending in “3,” a formal task asks for a proof of the encoded arithmetic claim. The LLM proposes tactics; Lean checks them. On harder theorems, several valid next steps may exist, and the critic estimates which resulting proof states are promising enough to explore.

The division of work becomes:

LLM proposes → Lean checks the transition → cheap critic ranks unfinished states → search allocates more attempts → Lean verifies a completed proof.

The actor supplies creative candidates. The critic guides resource allocation. **The critic does not replace the proof checker.** Its potential saving is against expensive learned judges or repeated lookahead rollouts. A cheap head also does not eliminate the transformer forward pass needed for a previously unseen state.

The practical sequence should be:

1. **Finish and report the PPO comparison.** Preserve it as the evidence for the original goal.
2. **Start with already formalized, manageable Lean theorems and a proof-capable actor.** Validate tactic generation before introducing a critic; our current math model is not automatically a competent Lean model.
3. **Collect proof-state trajectories and refit small heads.** Reuse our feature extraction, ridge/LSTD fitting and audit infrastructure, but do not assume natural-language critic weights transfer. Predict success within a declared search budget; failure within that budget does not mean a theorem is unprovable.
4. **Compare search fairly.** Measure policy-only search, repeated independent attempts, ridge-guided search and LSTD-guided search under matched token, verifier-call and compute budgets. Split by theorem, report all failures, and keep a nonlinear small-head control.
5. **Only then add learning from verified proofs.** Search changes the data distribution; fitting LSTD to arbitrary search branches does not automatically preserve an on-policy TD interpretation. Establish the simplest reliable search result before adding policy updates or more elaborate exploration.

This pivot makes the critic’s decision directly observable: **does its ranking help find more verified proofs per unit of compute?** Formal checking also addresses a weakness of our current setting: a correct final number can conceal invalid reasoning.

Our evidence supports pursuing that experiment, not predicting its outcome. The lasting objective is broader than LSTD: reuse an LLM’s internal computation to make learning or search cheaper, while preserving trustworthy evaluation.